

Rome EVM Private

Rome

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Apr 28, 2025 - Jun 28, 2025

SOURCE CODE
🔗 rome-evm-private

ASSESSED COMMIT ID
🔗 e127dca

SEVERITY BREAKDOWN



Summary

Rome engaged Halborn to conduct a security assessment on their Rome EVM program beginning on April 28, 2025 and ending on June 27, 2025. The security assessment was scoped to the smart contracts provided in the GitHub repository rome-evm-private, commit hashes, and further details can be found in the Scope section of this report.

The Rome team is releasing their Rome EVM program Solana program. This program collectively allows for functioning as an EVM-compatible execution layer on the Solana blockchain, enabling the deployment and execution of Solidity smart contracts.

Assessment Summary

Halborn was provided 45 business days for the engagement and assigned one full-time security engineer to review the security of the Solana Programs in scope. The engineer is a blockchain and smart contract security expert with advanced smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security vulnerabilities within the codebase of the program(s).
- Verify the correctness of the core logic for EVM state management via Solana accounts (PDAs); atomic and iterative EVM transaction execution; core EVM opcode compatibility, including precompiles and `CREATE2`; administrative access control for chain registration; and overall state consistency and error handling logic.
- Assess access control mechanisms ensuring only authorized parties can perform sensitive actions.
- Evaluate the security of state management, including initialization, updates (where applicable), and potential data inconsistencies.
- Analyze the implementation and usage of relevant cryptographic or Solana-specific mechanisms.
- Identify potential edge cases or logical flaws that could lead to unexpected behavior, denial of service, or irrecoverable fund lockups.
- Assess adherence to Solana development best practices regarding security, resource management (rent and compute), and CPI handling.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the `Rome` `team`. The main ones were the following:

- `Replace the use of unwrap_or_default across the program to prevent error propagation issues`
- `Modify the ecrecover precompile to return a specification-compliant Ethereum address, ensuring compatibility.`
- `Avoid checking balances to emulate the initialization of key storage accounts.`
- `Prevent Denial-of-Service attacks by removing strict zero-balance checks during on-chain account creation.`
- `Make critical PDA addresses stable by removing dependencies on changeable keys like the upgrade authority.`
- `Ensure all errors are properly propagated during administrative functions to prevent inconsistent states.`

Scope

REPOSITORY



(a) Repository:  rome-evm-private

(b) Assessed Commit ID:  e127dca5266ad907da3dec8a020299e2bc378bdc

(c) Items in scope:

- ✓  emulator 28 files
 - ✓  src 27 files
 - ✓  api 16 files
 - <> confirm_tx_iterative.rs Rust
 - <> deposit.rs Rust
 - <> do_tx_holder_iterative.rs Rust
 - <> do_tx_holder.rs Rust
 - <> do_tx_iterative.rs Rust
 - <> do_tx.rs Rust
 - <> eth_call.rs Rust
 - <> eth_estimate_gas.rs Rust
 - <> eth_get_balance.rs Rust
 - <> eth_get_code.rs Rust
 - <> eth_get_storage_at.rs Rust
 - <> eth_get_tx_count.rs Rust
 - <> get_rollups.rs Rust
 - <> mod.rs Rust
 - <> reg_owner.rs Rust
 - <> transmit_tx.rs Rust
 - ✓  context 4 files
 - <> atomic.rs Rust
 - <> iterative_lock.rs Rust
 - <> iterative.rs Rust
 - <> mod.rs Rust
 - <> allocate.rs Rust
 - <> entrypoint.rs Rust
 - <> lib.rs Rust
 - <> origin.rs Rust
 - <> state.rs Rust
 - <> stubs.rs Rust
 - <> vm_eth_call.rs Rust
 - <> Cargo.toml Toml
- ✓  program/src 74 files
 - ✓  accounts 14 files
 - <> account_state.rs Rust
 - <> account_type.rs Rust
 - <> code.rs Rust
 - <> holder.rs Rust

- <> lock.rs Rust
- <> mod.rs Rust
- <> owner_info.rs Rust
- <> ro_lock.rs Rust
- <> slot.rs Rust
- <> state_holder.rs Rust
- <> storage.rs Rust
- <> tx_holder.rs Rust
- <> valids.rs Rust
- <> ver.rs Rust
- ✓  api 8 files
 - <> deposit.rs Rust
 - <> do_tx_holder_iterative.rs Rust
 - <> do_tx_holder.rs Rust
 - <> do_tx_iterative.rs Rust
 - <> do_tx.rs Rust
 - <> mod.rs Rust
 - <> reg_owner.rs Rust
 - <> transmit_tx.rs Rust
- ✓  context 4 files
 - <> atomic.rs Rust
 - <> iterative_lock.rs Rust
 - <> iterative.rs Rust
 - <> mod.rs Rust
- ✓  non_evm 10 files
 - <> aspl_token_ix.rs Rust
 - <> aspl_token.rs Rust
 - <> aux.rs Rust
 - <> mod.rs Rust
 - <> non_evm_state.rs Rust
 - <> spl_token_ix.rs Rust
 - <> spl_token.rs Rust
 - <> system_ix.rs Rust
 - <> system.rs Rust
 - <> withdraw.rs Rust
- ✓  precompile 10 files
 - <> blake2f.rs Rust
 - <> ecadd.rs Rust
 - <> ecmul.rs Rust
 - <> ecpairing.rs Rust
 - <> ecrecover.rs Rust
 - <> identity.rs Rust
 - <> mod.rs Rust
 - <> modexp.rs Rust
 - <> ripemd_160.rs Rust
 - <> sha2_256.rs Rust
- ✓  state 11 files
 - <> allocate.rs Rust

```
<> aux.rs Rust
<> base.rs Rust
<> handler.rs Rust
<> info.rs Rust
<> journal.rs Rust
<> journaled_state.rs Rust
<> mod.rs Rust
<> origin.rs Rust
<> pda.rs Rust
<> state.rs Rust
✓ 📁 tx 6 files
  <> deposit.rs Rust
  <> eip1559.rs Rust
  <> eip2930.rs Rust
  <> legacy.rs Rust
  <> mod.rs Rust
  <> tx.rs Rust
✓ 📁 vm 5 files
  <> mod.rs Rust
  <> snapshot.rs Rust
  <> vm_atomic.rs Rust
  <> vm_iterative.rs Rust
  <> vm.rs Rust
<> alloc.rs Rust
<> assert.rs Rust
<> config.rs Rust
<> entrypoint.rs Rust
<> error.rs Rust
<> lib.rs Rust
```

Out-of-Scope: Third party dependencies.

REMEDATION COMMIT ID:



```
✦ ba5f6035239e49d637898b7273d0a0f1fee36952
✦ 571e283cda9c645c9c207cb913064292e57d516c
✦ 122052f8c6f3039438bc230d0fa2fe62d71a6016
✦ aa7c031e83ffa0b90568ea077e49fa050be53c2a
✦ c6d68c77356e5d2c9f8100c1075f67c8f37336fe
✦ 0afa22cfc857a9b8bba1804bd9eb18045ada8df4
✦ e918816dba4bb7c4dd69b851a7ee37314f2edcb6
```

Out-of-Scope: New features/implementations after the remediation commit IDs.

Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Improper Error Handling in State Handler Masks Missing Accounts	 Critical	9.4	SOLVED 07/16/2025
HAL-002	ecrecover Precompile Returns Incorrect Value	 Medium	5.0	SOLVED 07/15/2025
HAL-003	Denial-of-Service in Non-EVM System Precompile Emulation	 Low	3.1	RISK ACCEPTED 07/16/2025
HAL-004	Denial-of-Service by Pre-funding Program Derived Accounts	 Low	2.5	SOLVED 07/14/2025
HAL-005	Unstable PDA Address Generation Using Upgrade Authority as a Seed	 Low	2.0	SOLVED 07/16/2025
HAL-006	Incomplete Error Propagation in reg_owner Leads to State Inconsistency	 Low	2.0	SOLVED 07/16/2025
HAL-007	Hardcoded Upgrade Authority Key Impedes Key Rotation	 Informational	—	PARTIALLY SOLVED 07/16/2025
HAL-008	Precompile Addresses Are Not Uniquely Verified at Compile Time	 Informational	—	PARTIALLY SOLVED 07/16/2025
HAL-009	Misleading Error Type in info_sol_wallet Function Obscures Root Cause	 Informational	—	SOLVED 07/15/2025

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.